

Set 11: Product Category Analysis

Dataset: Product Categories

Category	Sales (in \$)
Electronics	50000
Clothing	35000
Appliances	40000

1. Generate a funnel chart to analyze the sales conversion process for each product category. Label the stages and title the chart.

R code:

```
library(plotly)

cat_data <- data.frame(
  Category = c("Electronics", "Appliances", "Clothing"),
  Sales = c(50000, 40000, 35000)
)

plot_ly(
  type = "funnel",
  y = cat_data$Category,
  x = cat_data$Sales,
  textinfo = "value+percent initial"
) %>%
  layout(title = "Sales Conversion Funnel by Product Category")
```

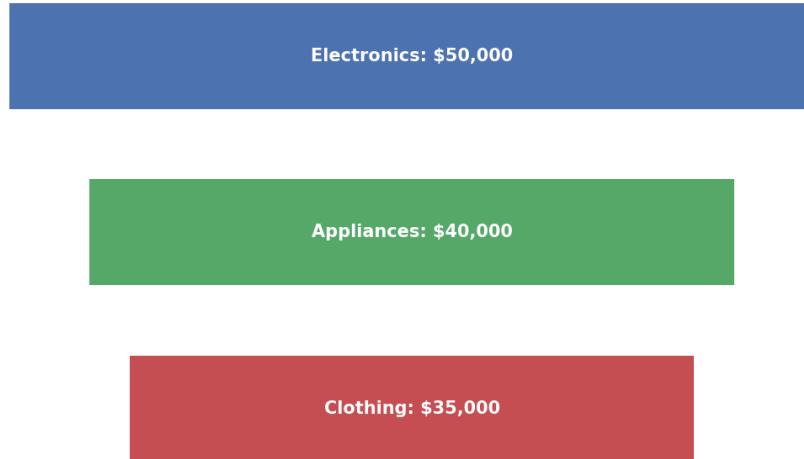
Python code:

```
import plotly.graph_objects as go

cat_data = cat_data.sort_values("Sales", ascending=False)

fig = go.Figure(go.Funnel(
  y = cat_data["Category"],
  x = cat_data["Sales"],
  textinfo = "value+percent initial"
))
fig.update_layout(title="Sales Conversion Funnel by Product Category")
fig.show()
```

Sales Conversion Funnel by Product Category



Interpretation: Electronics leads the conversion funnel with the highest sales (\$50,000), followed by Appliances (\$40,000) and Clothing (\$35,000), indicating Electronics converts best relative to the other categories.

2. Build a table to display the sales data for each product category. Label the table elements.

R code:

```
category <- data.frame(  
  Category = c("Electronics", "Clothing", "Appliances"),  
  Sales = c(50000, 35000, 40000)  
)  
print(category)
```

Python code:

```
import pandas as pd  
  
category = pd.DataFrame({  
  "Category": ["Electronics", "Clothing", "Appliances"],  
  "Sales": [50000, 35000, 40000]  
})  
print(category)
```

Category	Sales (in \$)
Electronics	50,000
Clothing	35,000
Appliances	40,000

3. Develop a Tableau dashboard combining the pie chart, funnel chart, and the table for interactive exploration of product category data.

Tableau steps:

1. Connect Tableau to the Product Categories dataset (Data > Connect > Text/Excel file).
2. Build the Pie Chart: drag Category to Color, Sales to Angle, choose the Pie mark type.
3. Build the Funnel Chart: drag Category to Rows (sorted descending by Sales), Sales to Columns, use a bar mark and apply the 'Funnel' custom shape/step-line technique, or use a third-party funnel chart extension.
4. Build the Table: create a new worksheet, drag Category and Sales to the view, use 'Show Me > Text Table'.
5. Create a new Dashboard, drag all three worksheets (Pie Chart, Funnel Chart, Table) onto the dashboard canvas.

6. Add a Category filter and set it to apply to all three worksheets, enabling interactive filtering (click a pie slice to filter the funnel and table).
7. Add titles, a dashboard title ('Product Category Sales Dashboard'), and arrange with a Tiled or Floating layout for clarity.
8. Publish to Tableau Server/Public or export as a packaged workbook (.twbx) for sharing.

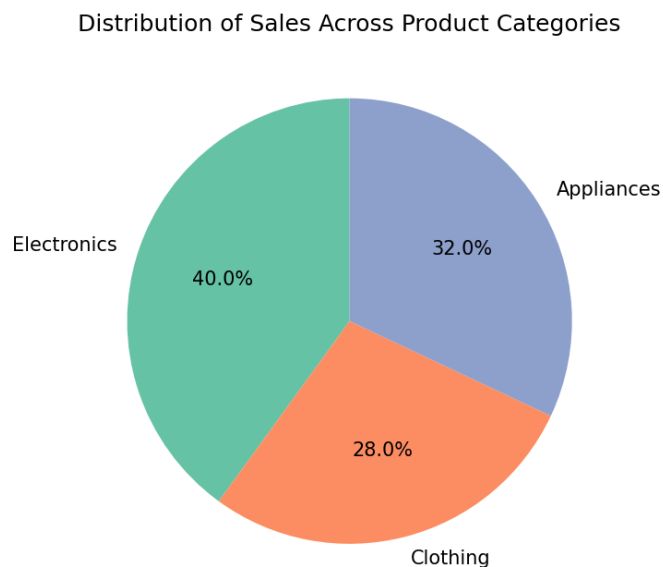
4. Create a pie chart to represent the distribution of sales across product categories. Include labels.

R code:

```
ggplot(category, aes(x = "", y = Sales, fill = Category)) +  
  geom_bar(stat = "identity", width = 1) +  
  coord_polar("y") +  
  labs(title = "Distribution of Sales Across Product Categories") +  
  theme_void()
```

Python code:

```
plt.figure(figsize=(5,5))  
plt.pie(category["Sales"], labels=category["Category"], autopct="%1.1f%%", startangle=90)  
plt.title("Distribution of Sales Across Product Categories")  
plt.show()
```



Set 12: Website Traffic Analysis

Dataset: Website Traffic

Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%

1. Create a line chart to visualize the trend in daily page views over time. Label the axes and title the chart.

R code:

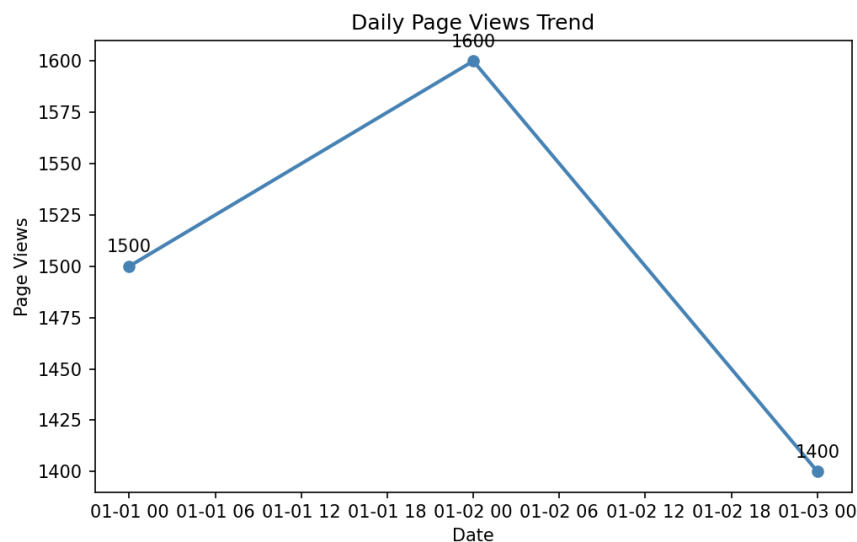
```
web$Date <- as.Date(web$Date)

ggplot(web, aes(x = Date, y = PageViews, group = 1)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(color = "steelblue", size = 3) +
  labs(
    title = "Daily Page Views Trend",
    x = "Date",
    y = "Page Views"
  ) +
  theme_minimal()
```

Python code:

```
web["Date"] = pd.to_datetime(web["Date"])

plt.figure(figsize=(7,4.5))
plt.plot(web["Date"], web["PageViews"], marker="o", color="steelblue")
plt.title("Daily Page Views Trend")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.show()
```



Interpretation: Page views rose from 1,500 to 1,600 between Jan 1 and Jan 2, then dropped to 1,400 on Jan 3, suggesting a short-lived spike in traffic mid-week.

2. Generate a bar chart showing the top N days with the highest click-through rates. Label the chart elements.

R code:

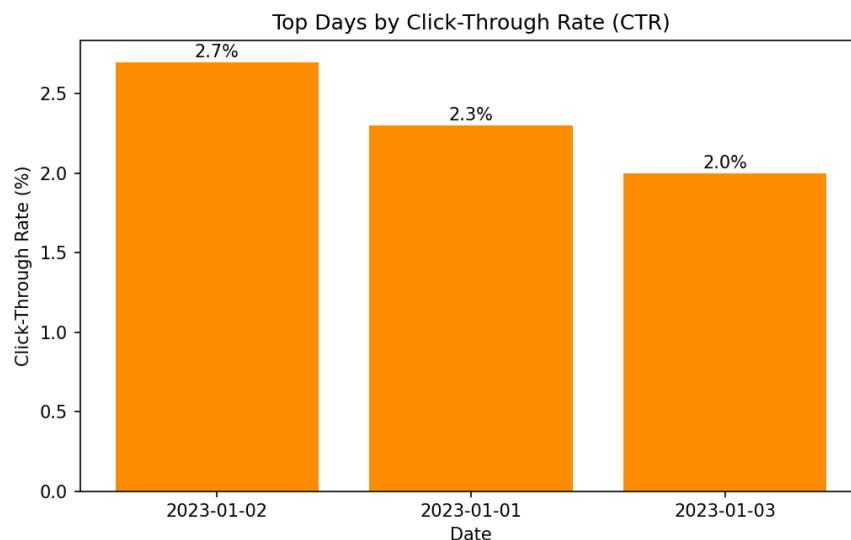
```
top_days <- web %>% arrange(desc(CTR)) %>% head(3)

ggplot(top_days, aes(x = reorder(as.character(Date), -CTR), y = CTR)) +
  geom_bar(stat = "identity", fill = "darkorange") +
  labs(
    title = "Top Days by Click-Through Rate (CTR)",
    x = "Date",
    y = "Click-Through Rate (%)"
  ) +
  theme_minimal()
```

Python code:

```
top_days = web.sort_values("CTR", ascending=False).head(3)

plt.figure(figsize=(7,4.5))
plt.bar(top_days["Date"].dt.strftime("%Y-%m-%d"), top_days["CTR"], color="darkorange")
plt.title("Top Days by Click-Through Rate (CTR)")
plt.xlabel("Date")
plt.ylabel("Click-Through Rate (%)")
plt.show()
```



3. Build a table to show the website traffic data for each day. Label the table elements.

R code:

```
web <- data.frame(
  Date = c("2023-01-01", "2023-01-02", "2023-01-03"),
  PageViews = c(1500, 1600, 1400),
  CTR = c("2.3%", "2.7%", "2.0%")
)
print(web)
```

Python code:

```
web = pd.DataFrame({
  "Date": ["2023-01-01", "2023-01-02", "2023-01-03"],
  "PageViews": [1500, 1600, 1400],
  "CTR": ["2.3%", "2.7%", "2.0%"]
})
print(web)
```

Date	Page Views	Click-through Rate
------	------------	--------------------

2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%

4. Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of website traffic data.

Tableau steps:

1. Connect to the Website Traffic dataset and ensure Date is set to a Date data type.
2. Build the Line Chart: drag Date to Columns, Page Views to Rows, select the Line mark type.
3. Build the Bar Chart: create a new worksheet, drag Date to Columns and CTR to Rows, sort descending, and use a Top N filter (e.g. Top 3) on CTR.
4. Build the Table: new worksheet with Date, Page Views, and CTR as a Text Table (Show Me > Text Table).
5. Create a new Dashboard and drag the Line Chart, Bar Chart, and Table onto the canvas.
6. Add a Date filter/highlight action so selecting a date on the line chart highlights the corresponding row in the table and bar chart.
7. Format with a dashboard title ('Website Traffic Dashboard'), legends, and consistent color coding.
8. Publish or save as a packaged workbook (.twbx).

Set 13: Geographic Data Analysis

Dataset: Geographic Data

City	Population	Avg. Temperature	Elevation
City A	500000	75	1000
City B	700000	68	800
City C	600000	80	1200

1. Create a map chart to visualize the distribution of cities on a geographic map. Label the map elements.

Note: the dataset does not include latitude/longitude, so illustrative coordinates are used below to demonstrate the mapping technique. Replace with actual coordinates for real analysis.

R code:

```
library(ggplot2)
library(maps)

geo <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  Elevation = c(1000, 800, 1200),
  Lat = c(34.05, 40.71, 29.76),
  Lon = c(-118.24, -74.00, -95.36)
)

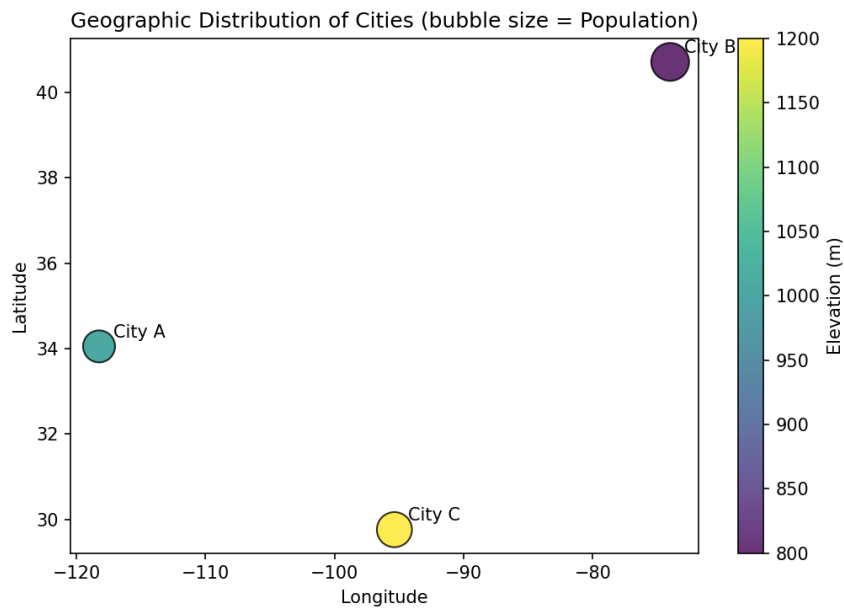
usa <- map_data("state")

ggplot() +
  geom_polygon(data = usa, aes(x = long, y = lat, group = group),
    fill = "gray95", color = "white") +
  geom_point(data = geo, aes(x = Lon, y = Lat, size = Population, color = Elevation)) +
  geom_text(data = geo, aes(x = Lon, y = Lat, label = City), vjust = -1) +
  labs(
    title = "Geographic Distribution of Cities",
    x = "Longitude", y = "Latitude",
    size = "Population", color = "Elevation (m)"
  ) +
  theme_minimal()
```

Python code:

```
geo = pd.DataFrame({
  "City": ["City A", "City B", "City C"],
  "Population": [500000, 700000, 600000],
  "Elevation": [1000, 800, 1200],
  "Lat": [34.05, 40.71, 29.76],
  "Lon": [-118.24, -74.00, -95.36]
})

plt.figure(figsize=(7,5))
sc = plt.scatter(geo["Lon"], geo["Lat"], s=geo["Population"]/1500,
  c=geo["Elevation"], cmap="viridis", alpha=0.8, edgecolor="black")
for _, r in geo.iterrows():
  plt.annotate(r["City"], (r["Lon"], r["Lat"]), textcoords="offset points", xytext=(8,5))
plt.title("Geographic Distribution of Cities (bubble size = Population)")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.colorbar(sc, label="Elevation (m)")
plt.show()
```



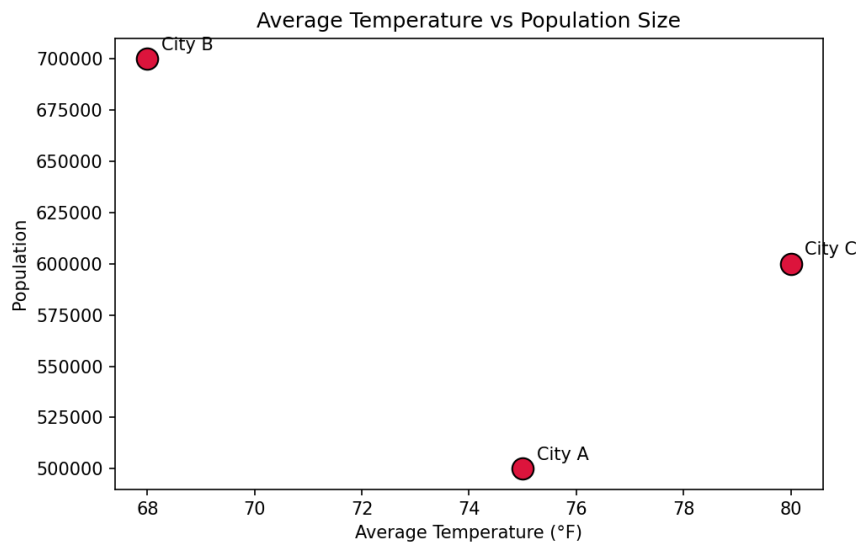
2. Generate a scatter plot to explore the relationship between average temperature and population size. Explain any insights.

R code:

```
ggplot(geo_full, aes(x = AvgTemp, y = Population, label = City)) +
  geom_point(size = 4, color = "crimson") +
  geom_text(vjust = -1) +
  labs(
    title = "Average Temperature vs Population Size",
    x = "Average Temperature (F)",
    y = "Population"
  ) +
  theme_minimal()
```

Python code:

```
plt.figure(figsize=(7,4.5))
plt.scatter(geo_full["AvgTemp"], geo_full["Population"], s=150, color="crimson", edgecolor="black")
for _, r in geo_full.iterrows():
    plt.annotate(r["City"], (r["AvgTemp"], r["Population"]), textcoords="offset points", xytext=(8,5))
plt.title("Average Temperature vs Population Size")
plt.xlabel("Average Temperature (F)")
plt.ylabel("Population")
plt.show()
```

Insight: With only three cities, no strong linear relationship is evident — City B has the highest population (700,000) but the lowest average temperature (68°F), while City C has the highest temperature (80°F) with a mid-range population, suggesting temperature alone does not explain population size in this small sample.

3. Build a table to display the geographic data for each city. Label the table elements.

R code:

```
geo_full <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  AvgTemp = c(75, 68, 80),
  Elevation = c(1000, 800, 1200)
)
print(geo_full)
```

Python code:

```
geo_full = pd.DataFrame({
  "City": ["City A", "City B", "City C"],
  "Population": [500000, 700000, 600000],
  "AvgTemp": [75, 68, 80],
  "Elevation": [1000, 800, 1200]
})
print(geo_full)
```

City	Population	Avg. Temperature	Elevation
City A	500,000	75	1,000
City B	700,000	68	800
City C	600,000	80	1,200

4. Develop a Tableau dashboard combining the map chart, scatter plot, and the table for interactive exploration of geographic data.

Tableau steps:

1. Connect to the Geographic Data dataset. If City is recognized as a geographic role, Tableau will auto-generate Latitude/Longitude; otherwise, add custom Lat/Lon fields.
2. Build the Map Chart: drag Longitude to Columns, Latitude to Rows (or double-click the City geo field), set mark type to Map/Circle, drag Population to Size and Elevation to Color.

3. Build the Scatter Plot: new worksheet, drag Avg. Temperature to Columns, Population to Rows, City to Detail/Label, use the Circle mark type.
4. Build the Table: new worksheet showing City, Population, Avg. Temperature, and Elevation as a Text Table.
5. Create a Dashboard and drag the Map, Scatter Plot, and Table onto the canvas.
6. Add a City filter action so clicking a city on the map highlights/filters it in the scatter plot and table.
7. Add a dashboard title ('Geographic Data Exploration Dashboard') and legends for Population/Elevation.
8. Publish or export as a packaged workbook (.twbx).

Set 14: Survey Results Analysis

Dataset: Survey Results

Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

1. Create a stacked bar chart to display the distribution of answers for Question 1. Label the chart elements.

R code:

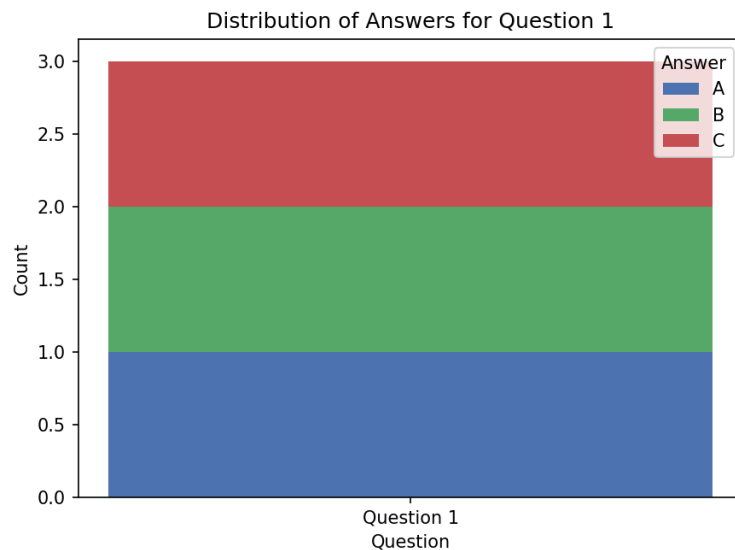
```
q1_data <- as.data.frame(table(survey$Question1))
colnames(q1_data) <- c("Answer", "Count")
q1_data$Question <- "Question 1"

ggplot(q1_data, aes(x = Question, y = Count, fill = Answer)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Distribution of Answers for Question 1",
    x = "Question",
    y = "Count",
    fill = "Answer"
  ) +
  theme_minimal()
```

Python code:

```
q1_counts = survey["Question1"].value_counts().sort_index()

plt.figure(figsize=(6,4.5))
bottom = 0
for answer, count in q1_counts.items():
    plt.bar("Question 1", count, bottom=bottom, label=answer)
    bottom += count
plt.title("Distribution of Answers for Question 1")
plt.xlabel("Question")
plt.ylabel("Count")
plt.legend(title="Answer")
plt.show()
```



2. Generate a radar chart to visualize the overall pattern of responses across all three questions.

R code:

```
library(fmsb)

options <- c("A", "B", "C", "D")
counts <- sapply(survey[,c("Question1", "Question2", "Question3")],
                 function(col) table(factor(col, levels = options)))
counts <- as.data.frame(t(counts))

radar_data <- rbind(rep(3,4), rep(0,4), counts)
colnames(radar_data) <- options

radarchart(radar_data,
            title = "Radar Chart: Response Pattern Across Questions",
            pcol = c("blue", "green", "red"),
            plwd = 2)
legend("topright", legend = rownames(counts), col = c("blue", "green", "red"), lty = 1, lwd = 2)
```

Python code:

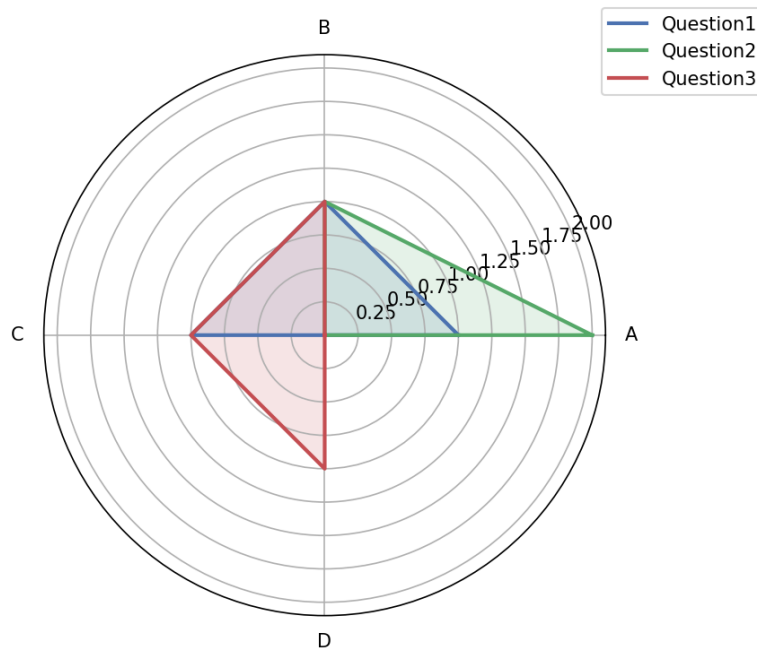
```
import numpy as np

options = ["A", "B", "C", "D"]
questions = ["Question1", "Question2", "Question3"]
counts = pd.DataFrame({q: survey[q].value_counts().reindex(options, fill_value=0) for q in questions})

angles = np.linspace(0, 2*np.pi, len(options), endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(6,6), subplot_kw=dict(polar=True))
for q in questions:
    vals = counts[q].tolist()
    vals += vals[:1]
    ax.plot(angles, vals, label=q, linewidth=2)
    ax.fill(angles, vals, alpha=0.15)
ax.set_xticks(angles[:-1])
ax.set_xticklabels(options)
ax.set_title("Radar Chart: Response Pattern Across Questions", y=1.1)
ax.legend(loc="upper right", bbox_to_anchor=(1.3,1.1))
plt.show()
```

Radar Chart: Response Pattern Across Questions



Interpretation: 'A' is the most frequent response for Question 2, while responses are more evenly spread for Questions 1 and 3, with 'D' appearing only once across all questions (Question 3).

3. Build a table to show the survey response data for each respondent. Label the table elements.

R code:

```
survey <- data.frame(
  Respondent = c(1,2,3),
  Question1 = c("A", "B", "C"),
  Question2 = c("B", "A", "A"),
  Question3 = c("C", "D", "B")
)
print(survey)
```

Python code:

```
survey = pd.DataFrame({
  "Respondent": [1,2,3],
  "Question1": ["A", "B", "C"],
  "Question2": ["B", "A", "A"],
  "Question3": ["C", "D", "B"]
})
print(survey)
```

Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

4. Develop a Tableau dashboard combining the stacked bar chart, radar chart, and the table for interactive exploration of survey responses.

Tableau steps:

1. Connect to the Survey Results dataset and pivot Question1, Question2, Question3 into a long format (Data > Pivot) to get Question and Response columns.
2. Build the Stacked Bar Chart: drag Question to Columns, count of Respondent (CNT) to Rows, Response to Color, mark type Bar.
3. Build the Radar Chart: Tableau has no native radar chart, so use a polygon/path technique (compute X/Y coordinates per question-response pair) or a radar chart extension from the Tableau Extension Gallery.
4. Build the Table: new worksheet showing Respondent, Question1, Question2, Question3 as a Text Table.
5. Create a Dashboard and add the Stacked Bar Chart, Radar Chart, and Table.
6. Add a Respondent filter action so selecting a respondent in the table highlights their responses in the other views.
7. Add a dashboard title ('Survey Responses Dashboard') and format colors consistently for A/B/C/D across all views.
8. Publish or export as a packaged workbook (.twbx).

Set 15: Product Sales Analysis

Dataset: Monthly Product Sales

Product ID	Product Name	January Sales	February Sales	March Sales
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

1. Create a grouped bar chart to visualize the sales of each product for the first quarter. Label the chart elements.

R code:

```
sales_long <- pivot_longer(sales, cols = c(January, February, March),
                           names_to = "Month", values_to = "Sales")
sales_long$Month <- factor(sales_long$Month, levels = c("January", "February", "March"))

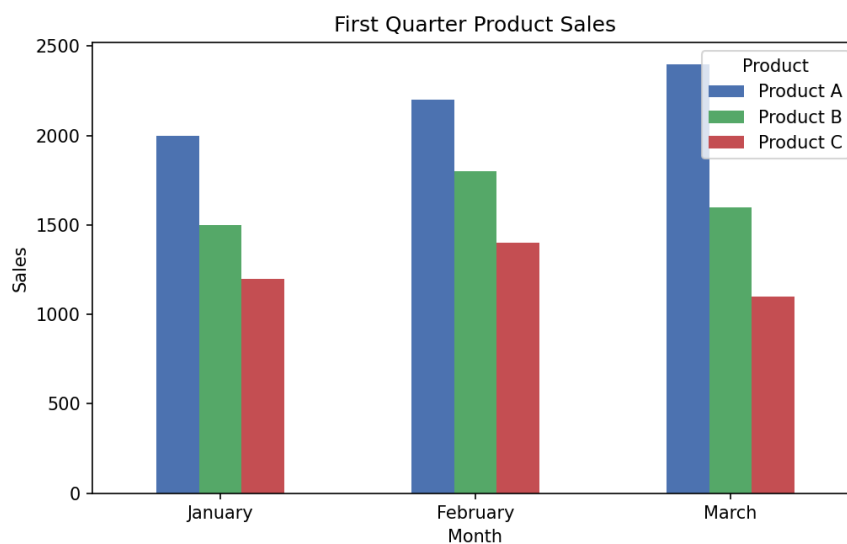
ggplot(sales_long, aes(x = Month, y = Sales, fill = Product)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "First Quarter Product Sales",
    x = "Month",
    y = "Sales",
    fill = "Product"
  ) +
  theme_minimal()
```

Python code:

```
sales_long = sales.melt(id_vars=["ProductID", "Product"],
                       value_vars=["January", "February", "March"],
                       var_name="Month", value_name="Sales")

pivot = sales_long.pivot(index="Month", columns="Product", values="Sales")
pivot = pivot.reindex(["January", "February", "March"])

pivot.plot(kind="bar", figsize=(7,4.5))
plt.title("First Quarter Product Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend(title="Product")
plt.show()
```



2. Generate a stacked area chart to represent the overall sales trend for all products over the three months.

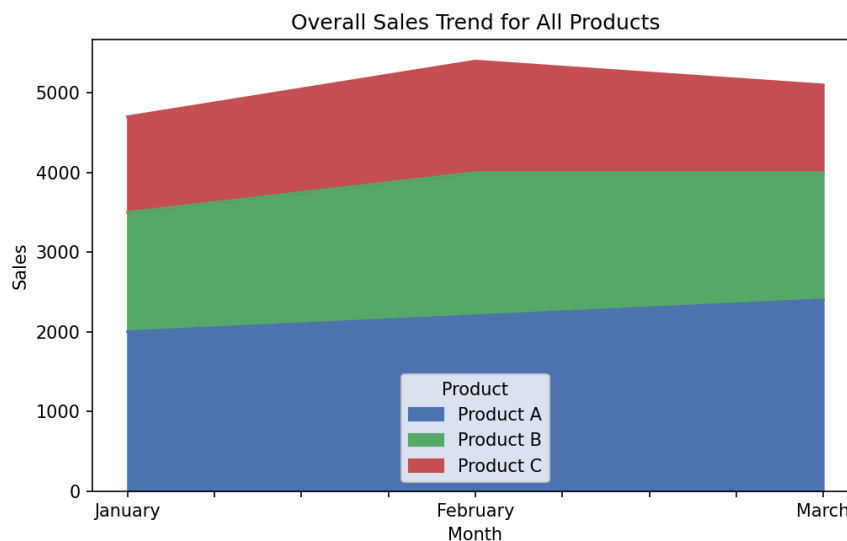
R code:

```
ggplot(sales_long, aes(x = Month, y = Sales, fill = Product, group = Product)) +
  geom_area(alpha = 0.7) +
  labs(
    title = "Overall Sales Trend for All Products",
    x = "Month",
    y = "Sales",
    fill = "Product"
  ) +
  theme_minimal()
```

Python code:

```
area = sales.set_index("Product")[["January", "February", "March"]].T

area.plot(kind="area", stacked=True, figsize=(7,4.5))
plt.title("Overall Sales Trend for All Products")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



3. Build a table to show the monthly sales data for each product. Label the table elements.

R code:

```
sales <- data.frame(  
  ProductID = c(1,2,3),  
  Product = c("Product A", "Product B", "Product C"),  
  January = c(2000,1500,1200),  
  February = c(2200,1800,1400),  
  March = c(2400,1600,1100)  
)  
print(sales)
```

Python code:

```
sales = pd.DataFrame({  
  "ProductID": [1,2,3],  
  "Product": ["Product A", "Product B", "Product C"],  
  "January": [2000,1500,1200],  
  "February": [2200,1800,1400],  
  "March": [2400,1600,1100]  
})  
print(sales)
```

Product ID	Product Name	January Sales	February Sales	March Sales
1	Product A	2,000	2,200	2,400
2	Product B	1,500	1,800	1,600
3	Product C	1,200	1,400	1,100

4. Develop a Tableau dashboard combining the grouped bar chart, stacked area chart, and the table for interactive exploration of sales data.

Tableau steps:

1. Connect to the Monthly Product Sales dataset and pivot January/February/March into Month and Sales columns (Data > Pivot).
2. Build the Grouped Bar Chart: drag Month to Columns, Sales to Rows, Product to Color, mark type Bar (side-by-side by dragging Product onto Columns as well for dodging).
3. Build the Stacked Area Chart: new worksheet, drag Month to Columns, Sales to Rows, Product to Color, mark type Area.
4. Build the Table: new worksheet showing Product, January, February, March as a Text Table.
5. Create a Dashboard and add the Grouped Bar Chart, Stacked Area Chart, and Table.
6. Add a Product filter/highlight action so selecting a product in one view highlights it across all views.
7. Add a dashboard title ('Product Sales Dashboard') and consistent color coding for Product A/B/C.
8. Publish or export as a packaged workbook (.twbx).